K-Factor Calculator

Explanation of `k-factor_calculator.pl` · Original author: Sara J. Beck (with Matthew Templeton), revised 3 January 2013 · Prepared 23 October 2026

Purpose

The script computes a **k-factor** for each solar observer in the AAVSO sunspot database. A k-factor is a personal scaling constant. It accounts for differences in telescope, seeing, and counting habits, so that `k × (observer's Wolf number)` lines up with the reference sunspot number, **Ra**.

Overall flow

1. Setup (lines 13–23)

- Loads `DBI` for database access and `Astro::Time` for converting dates to Julian Dates (JD).
- Connects to the MySQL database `solar` on `db.aavso.org` through `getdbh()` (lines 212–230).

2. Asking for the date range (lines 27–63)

- Asks for a start date as `MM-DD-YYYY` and converts it to a JD with `cal2mjd` → `mjd2jd`. The `0.5` argument puts the time at midday.
- Finds the last date that has an Ra value in `sunspot_counts` (`get_last_ra_date`, line 184), then keeps asking for an end date until:
  - the end date is no later than that last Ra date (a later date sets `$end_jd` to 0, forcing another try), and
  - the end date is **at least 100 days** after the start date.

3. Preparing the queries (lines 65–84)

Statement	Purpose
`select_observers`	Observers in `sun_data` with **≥ 100 observations** in the range (`revised = 0` only)
`select_wolf_numbers`	Every Wolf number `W` and date for one observer in the range
`select_ra`	Every Ra value (`count`) and JD from `sunspot_counts` in the range
`select_old_k`	The observer's current k-factor from `sun_obsconst`, for comparison

4. Loading data (lines 87–90)

- `get_observer_list` fills `@observer_list` with hashrefs `{A => initials, B => obs_count}`.
- `get_ra_numbers` fills `%ra_number`, mapping JD to Ra. Observer dates are later looked up in this hash.

5. Calculating each k-factor (lines 93–120)

For each observer, the script gets all their Wolf numbers and adds up two logs:

```
logRA   = Σ log10(Ra on that date)
logWolf = Σ log10(W observed)
logK    = (logRA − logWolf) / N
k       = 10^logK
```

This is the **geometric mean of Ra/W** across all of the observer's observations. Using logs keeps the ratio from being skewed by very large or very small single-day values.

Each result is printed next to the old k-factor, e.g. `ABC: 0.812 (143 obs) old k-factor: 0.795`, and an `observer<TAB>k` line is appended to `new_k-factors.txt` through `print_record` (line 233).

6. Helper subroutines

- `log10` uses the change-of-base formula, because Perl's `log` is the natural log.
- The `get_*` subroutines run a prepared query and gather the rows. They all use global variables.

Problems to know about

1. Database password in the source code (line 23). The credentials are hard-coded and therefore also in the git history. If they still work, they should be rotated and loaded from an environment variable or a config file.

2. Zero values get a made-up number (lines 103–112). The header comment flags this as a known issue. When W or Ra is 0, the script adds `0.1` to the log sum, which treats the value as ≈ 1.26. That skews the result, especially near solar minimum, when many observations are 0. The suggested fix is to skip those days; `N` must then count only the days actually used.

3. Dates with no Ra are treated as zero. If an observer's date has no entry in `%ra_number` (for example, a JD that doesn't match exactly), the lookup returns `undef`, which evaluates as 0, so the 0.1 substitute is used without any warning.

4. N comes from a different query. The divisor `$observation_count` comes from the observer-count query, not from the number of rows actually processed. The two agree today, but will not if zero days are skipped.

5. The output file is appended, never replaced. Each run adds another header and set of results to `new_k-factors.txt`.

6. Smaller code issues.

- `#!/usr/bin/perl -w` is on line 7, so it does not work as a shebang; it must be the first line.
- There is no `use strict`, and nearly everything is a global variable.
- `$a` and `$b` are used as ordinary variables, but Perl reserves them for `sort`.
- The date input is not validated.